



Universidad
de Alcalá

Escuela
Politécnica
Superior

The Linux Console

Sessions S02 (1/2) · S03 (2/2)

Universidad de Alcalá · EPS · 780007EN · Group 2E

Session structure

- **S02 (1/2)** — shell basics, navigation, files, permissions, editing (nano + vim)
- **S03 (2/2)** — command types, job control, process monitoring, I/O redirection, pipes, scripting

Installing Docker

We'll install this together in the lab — this slide is your reference.

- **macOS:** install **Docker Desktop** from <https://www.docker.com/products/docker-desktop/> (`.dmg`), then launch the Docker Desktop app once
- **Windows:** same Docker Desktop installer — requires **WSL2** enabled (the installer offers to set this up for you if it isn't already)
- **Linux:** install via your distro's package manager (e.g. `sudo apt install docker.io` on Ubuntu/Debian), then add yourself to the `docker` group so you don't need `sudo` every time: `sudo usermod -aG docker $USER` (log out and back in for it to take effect)

Verify it worked, on any OS:

```
docker --version
docker run hello-world
```

Starting the lab environment

```
docker run -it \  
  -v $(pwd):/workspace \  
  -w /workspace \  
  opobla/sog-lab:2026-2027
```

- `-v $(pwd):/workspace` — mounts your current directory inside the container, so files survive after it exits
- `-w /workspace` — starts you right there, not at some unrelated default directory
- Same image, same pattern (`make shell`) as Lab3/Lab4/Lab5 — you'll see this again

What is a shell, and why it matters

- A program that reads what you type, interprets it as a command, and asks the OS to run it
- `bash` (**B**ourne **A**gain **S**hell) — the default shell on virtually every Linux distro
- A brief lineage: Thompson shell (1971) → Bourne shell (1979) → bash
- Why it still matters with GUIs everywhere: **precise control** + **automation** — and it's the interface for almost every server/container/cloud machine running real workloads

The prompt and first commands

The prompt shows user, hostname, current directory, and privilege level (`$` vs `#`).

- `date` , `cal` — current date/time, calendar
- `whoami` , `uname` — who you are, what system this is
- `echo` — print text
- `type` — is this a shell builtin or an external executable?
- `clear` , `exit`

```
sog@sog-lab:/workspace$ whoami
sog
sog@sog-lab:/workspace$ date
Mon Sep 14 12:03:11 UTC 2026
```

History, completion, and help

- `history` — everything you've typed this session; `↑/↓` to navigate it
- `TAB` — completion (files, commands — press it, don't type it all)
- `man <command>` — the manual, organized in sections:
 - 1 → general commands, 2 → system calls, 3 → C library functions

```
$ man ls
LS(1)                  User Commands                  LS(1)
NAME
    ls - list directory contents
SYNOPSIS
    ls [OPTION]... [FILE]...
```

`q` to leave `man`, `/pattern` to search inside it.

The directory tree

```
/
├── bin      (executables)
├── etc      (configuration)
├── usr      (mount points)
├── mnt      (mount points)
├── home     (user directories)
├── dev      (devices, as files)
└── proc     (kernel info, as files)
```

Everything hangs off a single root `/` — no drive letters.

Absolute vs. relative paths

- **Absolute:** starts at `/`, doesn't depend on where you currently are
- **Relative:** starts at your current directory
- `.` current directory · `..` parent directory · `~` your home

Absolute vs. relative paths — a worked example

Starting from `/usr/local/bin`, both of these land you in `/usr/local/share`:

```
$ cd /usr/local/share    # absolute: always works, no matter where you started
$ cd ../share           # relative: up one level to /usr/local, then into share
```

Same destination — but the relative one only works **because** you started in `/usr/local/bin`. Type it from anywhere else and it takes you somewhere completely different.

Navigating the file system

- `pwd` — where am I?
- `ls` — what's here? (`-a` hidden files, `-l` detailed view)
- `cd` — change directory

```
$ pwd
/workspace
$ ls
notes.txt  scripts
$ cd scripts
$ pwd
/workspace/scripts
```

Reading `ls -l`

```
drwxr-xr-x  2 root    root    4096 Sep 12 08:42 subversion
-rw-rw-r--  1 parraman parraman 462 Nov 30 12:52 instr.txt
```

permissions · links · owner · group · size · last modified · name

Manipulating files and directories

- `cp` (`-r` for directories), `mv` (move/rename)
- `mkdir`, `rmdir` (empty directories only)
- `rm` (`-r` recursive, `-i` confirm, `-f` force — `rm -rf` without thinking is how you lose what you didn't mean to)

```
$ mkdir backup
$ cp notes.txt backup/
$ mv backup/notes.txt backup/notes-old.txt
$ rm -i backup/notes-old.txt
rm: remove regular file 'backup/notes-old.txt'? y
```

Wildcards and variables

- `*` matches any sequence of characters, `?` matches exactly one
- Shell variables (this session only) vs. environment variables (inherited by programs you launch): `$NAME` to use them — `HOME`, `SHELL`, `PATH` already come defined

```
$ ls *.txt           # every file ending in .txt
notes.txt  todo.txt
$ echo $HOME
/home/sog
```

Permissions

- Three classes (read, write, execute) × three subjects (owner, group, others)
- `chmod` changes them — symbolic (`chmod u+x file`) or octal (`chmod 755 file` , r=4/w=2/x=1 per subject)

Permissions — owner, groups, and `/etc/passwd`

- The **owner** is the user account that created the file (or was assigned it via `chown`) — the same account you log in as (`whoami`)
- Every user belongs to one or more **groups** — a set of users sharing some permission (e.g. a `teaching` group with write access to a shared folder, without giving that to everyone)
- That mapping lives in `/etc/passwd` (one line per user: name, UID, primary GID, home, shell) and `/etc/group`

```
-rw-rw-r-- 1 parraman parraman 462 Nov 30 12:52 instr.txt
           ↑ owner    ↑ group
```

Owner and group (both `parraman` here) can read+write; everyone else only reads.

Editing with nano

- Zero learning curve: the keys you need are shown on screen
- `^O` save · `^X` exit · `^W` search
- Good enough to edit anything from minute one

Editing with vim

vim works in **modes**, not free typing:

- **Normal** mode: move around, give commands (`dd` delete line, `yy` copy, `/pattern` search)
- **Insert** mode (`i` to enter): type text normally
- **Command-line** mode: `:w` save, `:q` quit, `:wq` both, `:q!` quit without saving

Run `vimtutor` (already installed) for a guided 25-30 min practice.

The role of `Esc` : it always takes you back to normal mode, no matter which mode you're in — the one reliable way to "get out" of wherever you are before giving the next command.

```
Esc → i (type your text) → Esc → :wq
```

Typing `:wq` while still in insert mode doesn't save or quit — it just types `:wq` literally into your text. Always press `Esc` first.

Checkpoint: end of S02 (1/2)

Types of commands

- **Executable** (a file on disk, findable with `which`)
- **Shell builtin** (`cd` , `echo` , ...)
- **Script** (a text file of commands, run by an interpreter)
- **Alias** (a short name for another command)

`type <command>` tells you which one you're dealing with.

```
$ type cd
cd is a shell builtin
$ type ls
ls is /usr/bin/ls
$ which ls
/usr/bin/ls
```

Job control

- Commands run in the **foreground** (blocks the prompt) or **background** (end the line with `&`)
- `jobs` — list background jobs
- `Ctrl+Z` — suspend the foreground job
- `fg` / `bg` — resume it in the foreground / background

```
$ sleep 100 &  
[1] 4821  
$ jobs  
[1]+  Running      sleep 100 &  
$ fg  
sleep 100          # now blocking the prompt again
```

Monitoring processes

- `ps` — list running processes
- `top` — live, continuously updating view
- `kill` — send a signal to a process (default: `SIGTERM`, "please terminate")

```
$ ps aux | grep sleep
sog    4821  0.0  0.0   2384  1536 pts/0  S   12:04   0:00 sleep 100
$ kill 4821
```

This is just enough to use the commands. What a signal actually is, or how the scheduler picks who runs, is `05-processes-threads` / `07-processes-lab` — coming soon.

Standard I/O and redirections

Every process is born with three channels: 0 = stdin, 1 = stdout, 2 = stderr (all default to the terminal).

- `<` redirect input from a file
- `>` / `>>` redirect output (overwrite / append)
- `2>` / `2>>` redirect errors
- `/dev/null` — a special file that discards anything written to it

```
$ echo "first run" > log.txt      # log.txt now contains just this line
$ echo "second run" >> log.txt    # appended, first line is still there
$ ls /nope 2> /dev/null          # the "No such file" error is silently discarded
```

Why three separate channels, not just one. The program itself doesn't know or care where its output goes — it just writes to "its" stdout (descriptor 1) and "its" stderr (descriptor 2), and the shell decides where each one lands *before* the program even starts. Keeping errors and normal output on separate channels is what lets you do this:

```
$ grep "TODO" *.c > found.txt 2> errors.txt
```

Pipes and filters

- `|` connects one command's stdout to the next one's stdin — chain as many as you like
- A **filter** reads stdin, writes stdout — the Unix philosophy: small programs, combined
- `sort`, `cut`, `grep`, `head / tail`, `uniq`, `wc`

```
$ ls -l | grep '^d' | wc -l # how many subdirectories are here?
```

Basic scripting (1/2)

```
#!/bin/bash
# shebang: tells the system which interpreter to use

echo "Hello, $1"    # $1 is the first argument
```

- `chmod +x script.sh` to make it executable
- `./script.sh argument` to run it — the `./` is needed because the current directory isn't normally in `$PATH`

Basic scripting (2/2)

```
#!/bin/bash
for f in *.txt; do
    if [ -s "$f" ]; then      # -s: file exists and is not empty
        wc -l "$f"
    fi
done
```

for / while loops, if conditionals, \$1 / @\$ / \$# for arguments — enough to automate a real repetitive task.

Checkpoint: end of S03 (2/2)



Live demo

- Build a long pipeline live, one filter at a time: `cat file | grep pattern | sort | uniq -c | sort`
`rn`
- `top` while something heavy runs in another terminal (`yes > /dev/null &`), then `kill` it — watch it disappear from the list
- Turn a command you typed by hand last session into a `.sh` script with a loop

Exercises to practice at home

- **E1** — Display the current date in `31-12-2019` format
- **E2** — Copy `/etc/passwd` to your working directory, once with an absolute path and once with a relative one
- **E3** — Save current disk usage to a file whose name includes the exact date/time
- **E4** — Count how many images are on `https://www.uah.es`
- **E5 (Create)** — Design your own small script that automates a repetitive task

Wrap-up

- Next: `03-os-structure` — kernel vs. user mode, system calls
- Keep practicing: `man bash`, `vimtutor`
- Reference: *The Linux Command Line* — http://linuxcommand.org/lc3_learning_the_shell.php